RCW-CIM: A Digital CIM-based LLM Accelerator with Read-Compute/Write

Yan-Cheng Guo[1], Tian-Sheuan Chang[1], Senior Member, IEEE, and Jian-Wei Su[2]
[1]*Institute of Electronics, National Yang Ming Chiao Tung University,* Hsinchu, Taiwan
[2]*Industrial Technology Research Institute,* Hsinchu, Taiwan

2026 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3315-7769-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCAS66217.2026.11562180

***Abstract*—Digital computing-in-memory (DCIM) has emerged as a promising solution for large language model (LLM) acceleration by minimizing data transfers between external DRAM and on-chip accelerators while maintaining high precision for superior accuracy. However, existing CIM architectures often overlook weight update latency, which becomes critical as LLM weights are far larger than a single CIM macro's capacity. To address this issue, this paper proposes a read-compute/write (RCW) architecture that effectively minimizes weight update latency, along with a nonlinear operator fusion that further mitigates dependency-induced latency. The proposed RCW reduces decoding computing latency by 21.59% on the Llama2-7B model. In addition, the nonlinear operator fusion mechanism achieves a 69.17% latency reduction through efficient partial accumulation and group-based approximation. Furthermore, a weight-stationary and output-column-stationary (WS-OCS) dataflow is introduced to reduce both external DRAM access and internal CIM weight updates—by 51.6% and 87.6%, respectively—during the prefill phase of 1024 tokens, leading to an overall 49.76% latency reduction. Fabricated using TSMC 22 nm CMOS technology and operating at 100 MHz, the proposed RCW-CIM achieves 3.28 TOPS and 42.3 TOPS/W, enabling 4.2 ms prefill latency and 26.87 decoded tokens per second for the INT4-weight Llama2 model with dual DDR5-6400 memory.**

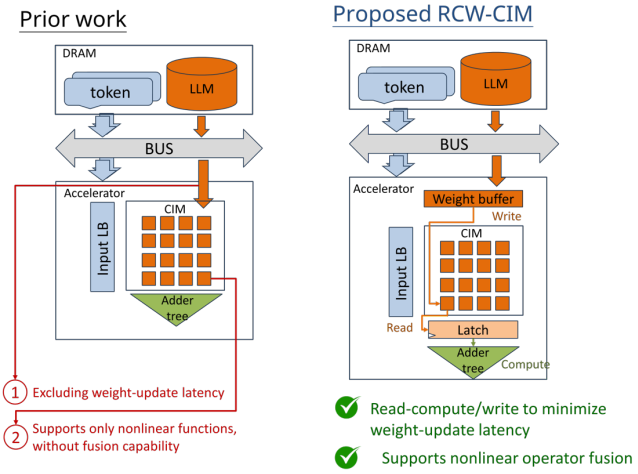


Fig. 1. Prior work [4], [5] typically ignores weight-update latency and does not optimize nonlinear-function execution. In contrast, the proposed RCW-CIM adopts a read-compute/write to minimize weight-update latency and supports nonlinear operator fusion to reduce dependencies in Softmax and RMSNorm operations.

I. Introduction

Large language models (LLMs) have gained popularity due to their high performance in natural language processing tasks [1]. To accommodate fine-tuning for diverse tasks, model sizes have grown substantially, enabling excellent accuracy across a wide range of applications [2]. However, the massive model weights significantly increase data transfer between DRAM and accelerators, and lower-precision representations are insufficient to maintain high accuracy. One promising approach is digital computing-in-memory (DCIM) [3], which combines computation and memory to reduce the weight transfer overhead while minimizing computational loss and preserving high precision.

However, prior DCIM work for LLMs [4] primarily focuses on reducing data alignment latency and encoding for weights or KV caches, without addressing weight update latency on the CIM—a critical performance bottleneck, especially given that LLM weights far exceed the size of a single CIM macro. Similarly, [5] implements nonlinear function computation within the CIM macro, but does not account for global latency of nonlinear operations and supports only 8-bit precision, which is insufficient for accurate nonlinear computation (Fig. 1).

To overcome these limitations, we propose RCW-CIM, which implements a read-compute/write (RCW) to minimize weight update latency and employs operator fusion for high-precision nonlinear functions to reduce global latency while maintaining high accuracy. RCW-CIM achieves 3.28 TOPS and 42.3 TOPS/W using TSMC 22 nm technology at 100 MHz, with a prefill latency of 4.2 ms and a decoding throughput of 26.87 tokens per second using dual DDR5-6400 memory.

The paper is organized as follows: Section II presents the RCW-CIM architecture and dataflow, Section III shows experimental results and comparisons, and Section IV concludes the paper.

II. Proposed Hardware Architecture and Data Flow

A. The overall architecture and CIM macro

Fig. 2 illustrates the system-level architecture of RCW-CIM, which integrates a WS-OCS scheduling unit optimized for weight-update latency, a high-performance nonlinear operator fusion controller, and eight CIM clusters. Each cluster comprises four energy-efficient CIM cores, a 64 KB input-reuse buffer, and a 64 KB partial-sum buffer.

Fig. 3 illustrates the proposed energy-efficient digital SRAM-based CIM macro, which supports FP16 precision for

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:06:30 UTC from IEEE Xplore. Restrictions apply.

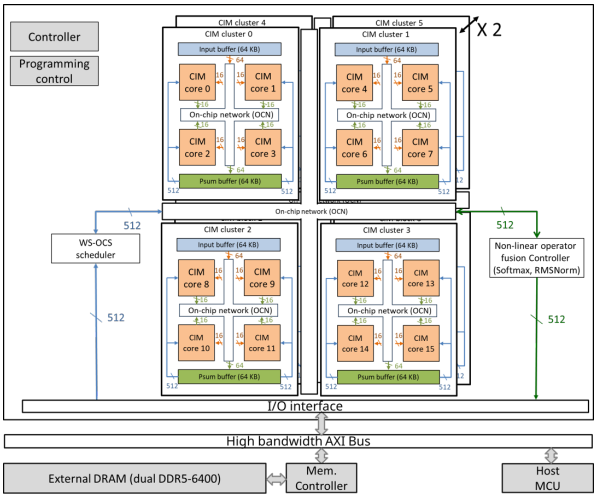


Fig. 2. The RCW-CIM overall architecture comprises 64 multi-CIM cores, a high-efficiency WS-OCS scheduler, and a nonlinear operator fusion controller supporting Softmax and RMSNorm.

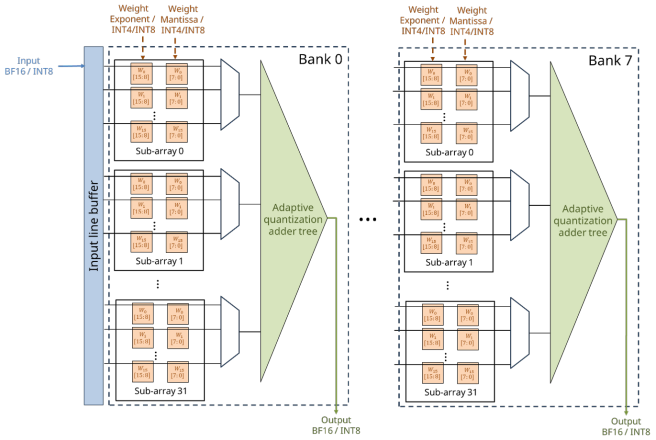


Fig. 3. The proposed energy-efficient digital SRAM-based CIM macro.

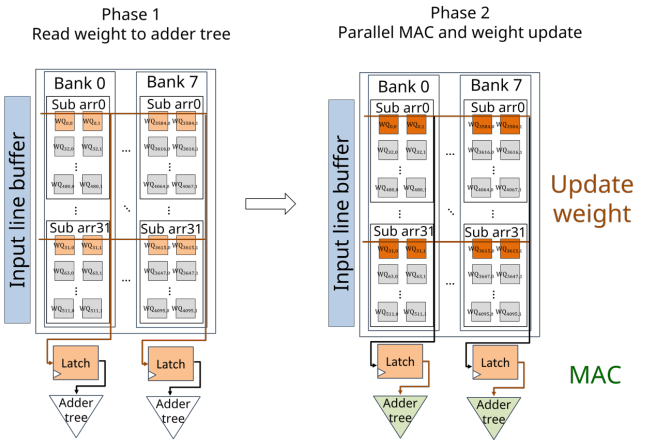


Fig. 4. Read-compute/write (RCW) in the CIM macro: Phase 1 reads weights into the adder tree, and Phase 2 performs parallel MAC and weight update.

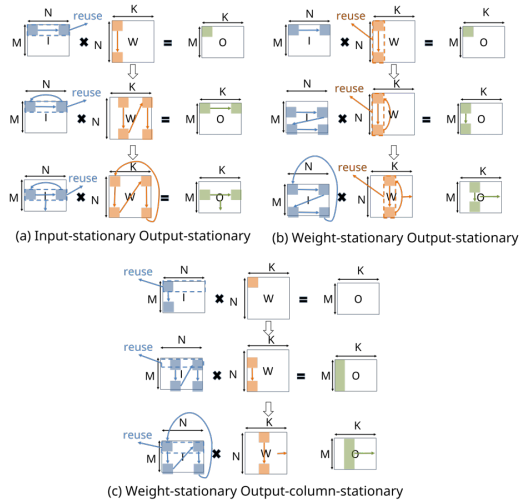


Fig. 5. Comparative analysis of the IS-OS, WS-OS, and WS-OCS dataflows, where the input, weight, and output matrices have dimensions $M \times N$, $N \times K$, and $M \times K$, respectively. The accelerator processes data in tiles of sizes $m \times n$ for input, $n \times k$ for weight, and $m \times k$ for output. (a) Input-stationary output-stationary dataflow (b) Weight-stationary output-stationary dataflow (c) Proposed Weight-stationary output-column-stationary dataflow

nonlinear function computation and a high-throughput dual INT4/INT8 computing mode. The CIM macro consists of eight banks, each capable of performing 32 MAC operations in parallel.

B. Low weight update latency with read-compute/write

To minimize LLM weight update latency, we propose a read-compute/write operation for the CIM macro as shown in Fig. 4 to overlap computing and weight update for lower latency. This performs a two-phase operation: in Phase 1, weights are read and latched for compact adder-tree computation; in Phase 2, MAC operations and weight updates execute concurrently. With this, weight update latency can be hided during computation. To further reduce WL decoding delay and precharge power, the selected WL remains active throughout both phases, enabling energy-efficient updates. Additionally, this approach supports low-latency transitions between nonlinear function execution and MAC computation via a deeply parallel weight-update and compute design.

C. Dataflow with Weight-stationary output-column-stationary (WS-OCS)

To reduce external DRAM bandwidth, digital AI accelerators have adopted two common dataflows, input-stationary output-stationary (IS-OS) and weight-stationary output-stationary (WS-OS), to maximize data reuse, as shown in Fig. 5. However, IS-OS and WS-OS are unsuitable for CIM architectures due to frequent weight updates. To address this, Fig. 5(c) presents the proposed weight-stationary output-column-stationary (WS-OCS) dataflow, where block weights are loaded and computed column-wise with input data, while partial sums are buffered internally. The input data is then

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:06:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DRAM ACCESS AND CIM UPDATE FOR IS, WS, IS-OS, WS-OS, AND WS-OCS DATAFLOWS

Dataflow	External DRAM			Internal CIM
	Input	Weight	Output	Weight update
IS	MN	$\frac{M}{m} \times NK$	$\frac{N}{n} \times MK$	$\frac{M}{m} \times NK$
WS	$\frac{K}{k} \times MN$	NK	$\frac{N}{n} \times MK$	NK
IS-OS [6]	MN	$\frac{M}{m} \times NK$	MK	$\frac{M}{m} \times NK$
WS-OS [6]	$\frac{K}{k} \times MN$	NK	MK	$\frac{M}{m} \times NK$
WS-OCS	$\frac{K}{k} \times (M-m)N$	NK	MK	NK

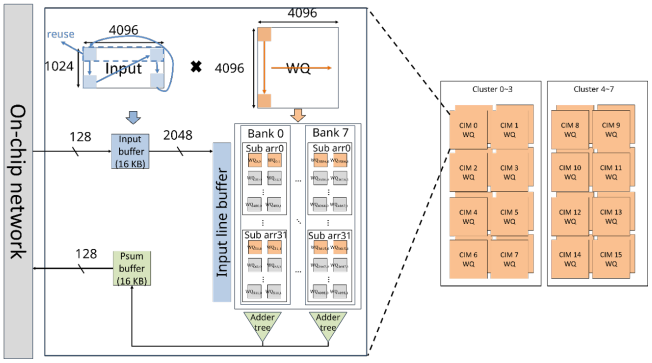


Fig. 6. Mapping of the WS-OCS dataflow onto the CIM macro.

scanned along the N dimension to compute column outputs, repeating this process for all weight blocks to produce the final output.

Table I summarizes external DRAM access and internal CIM weight-update behavior for IS, WS, IS-OS, WS-OS, and WS-OCS dataflows. To minimize external DRAM traffic, WS-OCS employs an input-reuse buffer, reducing input DRAM access from $\frac{K}{k} \times MN$ to $\frac{K}{k} \times (M-m)N$ compared with WS-OS. Moreover, WS-OCS markedly lowers CIM weight-update latency and volume (from $\frac{M}{m} \times NK$ to NK) and further reduces weight-buffer capacity by replacing weights only after all corresponding inputs are fully processed.

Fig. 6 illustrates the CIM macro mapping for LLM under the WS-OCS dataflow. In this scheme, Q weight blocks are distributed across the eight CIM clusters, while the input line buffer is shifted through the input-reuse buffer. The partial-sum buffer stores the intermediate column outputs, and the complete output results are obtained by scanning the input data along the N dimension and subsequently written back to DRAM.

D. Low-Latency nonlinear operator fusion in CIM Macro

Online Softmax and group RMSNorm have been proposed to reduce global dependency latency on systolic arrays [7]. However, CIM architectures still suffer from high power consumption and low efficiency when handling exponential and division operations. Thus, a 64-segment look-up table (LUT)-based group Softmax is adopted to reduce nonlinear

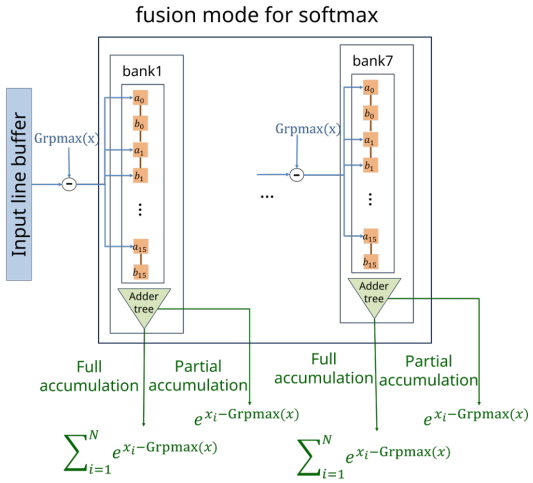


Fig. 7. Nonlinear group softmax fusion in the CIM macro supports full and partial accumulation to compute exponentials in parallel, improving overall efficiency.

dependency latency and prevent overflow and underflow, as shown in (1).

$$\text{Softmax}(x_i) \approx \frac{\text{LUT}_{64}(x_i - \max_{g\in G} x_g)}{\sum_{g\in G} \text{LUT}_{64}(x_g - \max_{g\in G} x_g)}, \quad i \in G \tag{1}$$

Prior CIM implementations [5] executed softmax inefficiently because they only supported full accumulation, which significantly reduced CIM utilization. To address this issue, we propose nonlinear operator fusion in the CIM macro, which not only supports full accumulation for computing the sum of exponentials, but also enables partial accumulation to perform exponentiation in parallel, thereby improving overall CIM efficiency.

Fig. 7 illustrates the group softmax fusion in the CIM macro. To minimize global dependencies, the input is first offset by the group maximum. The approximate coefficients a and b are then stored in the CIM macro using a 64-segment LUT. Finally, the adder tree performs both full and partial accumulation to efficiently compute the exponent sum and exponentiation. Additionally, high-precision FP16 is used to compute softmax, reducing accuracy loss.

RMSNorm [8] applies group RMS to reduce global latency. To further save computation time and mitigate accuracy loss, the synchronization to the global RMS is performed together with the γ scaling, as shown in (2).

$$\text{RMSNorm}(x_i) = \frac{x_i}{\sqrt{\frac{1}{|G_m|}\sum_{j\in G_m} x_j^2 + \epsilon}} \cdot \gamma_i, \quad i \in G_m \tag{2}$$

III. EXPERIMENTAL RESULT

RCW-CIM is implemented using TSMC 22 nm CMOS at 100 MHz, achieving 3.28 TOPS and 42.3 TOPS/W. The Llama2-7B model employs INT4 weights, INT8 activations, and FP16 for nonlinear functions. Table II summarizes performance comparisons. While [5] supports nonlinear functions,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:06:30 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF PERFORMANCE AND COMPARISON WITH OTHER DESIGNS.

	VLSI'25 [5]	VLSI'25 [4]	**This work**
Technology	28nm	28nm	**22nm**
Architecture	SRAM-based digital CIM	SRAM-based digital CIM	**SRAM-based digital CIM**
Macro size (KB)	12	128	**256**
Supply Voltage (V)	0.65	0.7	**0.8**
Frequency (MHz)	85-410	110-500	**100**
Input precision	INT4 / INT8	BF16	**INT8 / BF16**
Weight precision	INT4 / INT8	INT4 / INT8	**INT4 / INT8 / BF16**
Throughput (TOPS)	0.63 (0.63[1])	2.98 (5.96[1])	**3.28 (3.28[1])**
Energy efficiency (TOPS/W)	36.3 (30.5[2])	15.6 (30.4[2])	**42.3 (42.3[2])**
Model	LLaMa2-7B	LLaMa2-7B	**LLaMa2-7B**
Prefill latency (ms)	19.4	7.2	**4.2**
Decode token/s	-	8.51	**26.87**
Support nonlinear operation	✓(not support fusion)	-	**✓ (support fusion)**
Weight update optimization	-	-	✓

[1]Normalized operations (in equivalent 8-bit ops) = operations$\times \frac{\text{input precision}}{8\text{ bit}} \times \frac{\text{weight precision}}{8\text{ bit}}$

[2]Normalized energy efficiency (in equivalent 8-bit precision) = energy efficiency$\times \frac{\text{input precision}}{8\text{ bit}} \times \frac{\text{weight precision}}{8\text{ bit}} \times \left(\frac{\text{process}}{22\text{ nm}}\right) \times \left(\frac{\text{voltage}}{0.8\text{ V}}\right)^2$

it lacks nonlinear operator fusion and relies on INT8, causing higher precision loss. RCW-CIM applies nonlinear operator fusion to reduce nonlinear function latency and uses FP16 to maintain accuracy. Moreover, [5] and [4] do not consider weight update latency; RCW-CIM addresses this via RCW, minimizing latency. By combining WS-OCS dataflow with RCW, RCW-CIM achieves 4.2 ms prefill latency and 26.87 decoded tokens per second for the INT4-weight Llama2 model using dual DDR5-6400 memory.

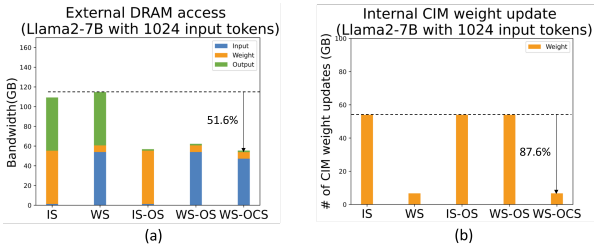


Fig. 8. (a) Compared with the WS dataflow, the WS-OCS scheme reduces external memory access by 51.6% during the prefill stage with 1024 input tokens. (b) The WS-OCS dataflow also decreases internal CIM weight-update operations by 87.6% compared with the IS-OS and WS-OS schemes.

Fig. 8 analyzes the impact of the proposed approaches. As shown in (a), WS-OCS reduces external DRAM access by 51.6% compared with WS via partial sums and input-reuse buffers. As shown in (b), WS-OCS further cuts internal CIM weight-update operations by 87.6% relative to IS-OS and WS-OS through column-wise weight mapping and N-dimensional input scanning.

Fig. 9 shows that the proposed techniques progressively reduce computation latency: WS-OCS cuts prefill latency by 49.76%, RCW reduces decode latency by 21.59%, and nonlinear operator fusion combined with group Softmax and RMSNorm further lowers decode latency by an additional 69.17%.

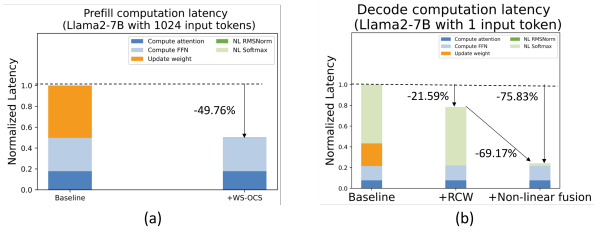


Fig. 9. The baseline employs multi-macro parallel weight updates to reduce weight-update latency. Through the proposed methods, performance is further improved: (a) The WS-OCS dataflow reduces computation latency by 49.76% during the prefill stage. (b) The RCW and nonlinear operator fusion modules further reduce latency by 75.83% during the decode stage.

IV. CONCLUSION

This paper presents RCW-CIM, achieving 3.28 TOPS and 42.3 TOPS/W using TSMC 22 nm CMOS technology at 100 MHz. RCW-CIM introduces the WS-OCS dataflow, which reduces external DRAM access by 51.6 % and internal CIM weight updates by 87.6 % during the prefill of 1024 tokens, leading to a 49.76 % reduction in prefill latency. Furthermore, the proposed RCW mitigates weight update overhead, reducing decode computing latency by 21.59 %. The nonlinear operator fusion technique further decreases decode latency by 69.17 % through partial accumulation, while group Softmax and RMSNorm synchronization minimize global latency. As a result, RCW-CIM achieves a prefill latency of 4.2 ms and a decoding throughput of 26.87 tokens/s for the INT4-weight Llama2-7B model with dual DDR5-6400 memory.

ACKNOWLEDGMENT

This work was supported by the National Science and Technology Council, Taiwan, under Grant 111-2622-8-A49-018-SB, 110-2221-E-A49-148-MY3, 113-2221-E-A49-078-MY3, and 113-2640-E-A49-005.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:06:30 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Radford *et al.*, "Improving language understanding by generative pre-training," 2018.

[2] T. B. Brown *et al.*, "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[3] Y.-D. Chih *et al.*, "16.4 an 89tops/w and 16.3tops/mm2 all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 252–254.

[4] Z. Wu *et al.*, "CELLA: A 28nm compute-memory co-optimized real-time digital CIM-based edge LLM accelerator with 1.78ms-response in prefill and 31.32 token/s in decoding," in *2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2025, pp. 1–3.

[5] Y. Wang *et al.*, "LLM-CIM: A 28nm 126.7TOPS/W input-LUT-based digital cim macro with reconfigurable matrix multiplication and nonlinear operation modes for LLMs," in *2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2025, pp. 1–3.

[6] T.-J. Li and T.-S. Chang, "An efficient data reuse with tile-based adaptive stationary for transformer accelerators," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025, pp. 1–5.

[7] J. Lin *et al.*, "Systolicattention: Fusing flashattention within a single systolic array," *arXiv preprint arXiv:2507.11331*, 2025.

[8] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:06:30 UTC from IEEE Xplore. Restrictions apply.